

Condition variables

Question: How would a thread check whether a condition is true before continuing its execution?

For eg:- a parent thread would want to know whether a child thread has completed before continuing. How to implement such a wait?

```
1 void *child(void *arg) {
2     printf("child\n");
3     // XXX how to indicate we are done?
4     return NULL;
5 }
6
7 int main(int argc, char *argv[]) {
8     printf("parent: begin\n");
9     pthread_t c;
10    Pthread_create(&c, NULL, child, NULL); // child
11    // XXX how to wait for child?
12    printf("parent: end\n");
13    return 0;
14 }
```

Expected output from the above code.

```
parent: begin
child
parent: end
```

A spin-based approach to get the output above

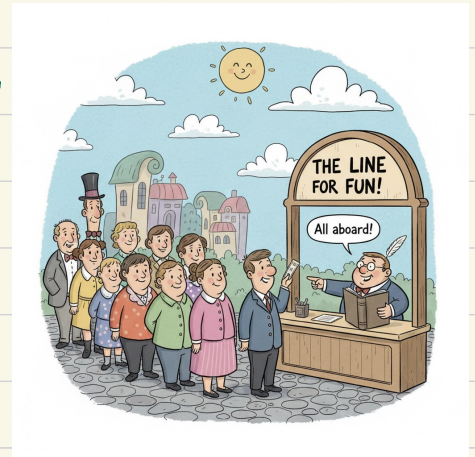
```
1 volatile int done = 0;
2
3 void *child(void *arg) {
4     printf("child\n");
5     done = 1; → a shared variable
6     return NULL;
7 }
8
9 int main(int argc, char *argv[]) {
10    printf("parent: begin\n");
11    pthread_t c;
12    Pthread_create(&c, NULL, child, NULL); // child
13    while (done == 0)
14        ; // spin
15    printf("parent: end\n");
16    return 0;
17 }
```

Problem with this approach
→ inefficient and wastes CPU cycles.

How to wait for a condition to become true efficiently?
→ using condition variable.

It is essentially a waiting queue for threads. It allows threads to pause execution until some condition in the program becomes true. Like, "wait here until called"

- threads check some condition (eg. is the buffer empty?)
- if the condition is not what they want, they wait in the queue the condition variable



- once some other thread changes the state (eg:- adds data to the buffer), it signals one or more waiting threads to wake up.

How to use it?

Declaration: `pthread_cond_t c;` // declares `c` as a condition variable
(properly initialize)

Two main operations: `wait()` and `signal()`

executed when a thread wishes to put itself to sleep if the condition is false

→ executed when a thread has changed something in the program and thus wants to wake a sleeping thread waiting on this condition.

```
pthread_cond_wait(pthread_cond_t *c, pthread_mutex_t *m);  
pthread_cond_signal(pthread_cond_t *c);
```

why there is a lock in the parameter?
→ will explain soon!

Parent waiting for child: using a condition variable

```
1  int done = 0;
2  pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
3  pthread_cond_t c = PTHREAD_COND_INITIALIZER;
4
5  void thr_exit() {
6      pthread_mutex_lock(&m);
7      done = 1;
8      pthread_cond_signal(&c);
9      pthread_mutex_unlock(&m);
10 }
11
12 void *child(void *arg) {
13     printf("child\n");
14     thr_exit();
15     return NULL;
16 }
17
18 void thr_join() {
19     pthread_mutex_lock(&m);
20     while (done == 0)
21         pthread_cond_wait(&c, &m);
22     pthread_mutex_unlock(&m);
23 }
24
25 int main(int argc, char *argv[]) {
26     printf("parent: begin\n");
27     pthread_t p;
28     pthread_create(&p, NULL, child, NULL);
29     thr_join();
30     printf("parent: end\n");
31     return 0;
32 }
```

shared flag indicating whether the child has finished.

mutex (lock) to protect access to done

condition variable

called by thread child when it finishes

→ signal the condition variable c → wakes up any thread waiting on c.

(in this case, parent thread)

→ to mark completion and signal the parent

parent waits:

atomically unlocks m and puts thread to sleep. But when it wakes up after being signaled, it relocks m before returning. That's why, we need to explicitly call unlock on the following line.

Let us focus once again on two important parts of code.

child thread (signaler)

```
void thr_exit() {  
    Pthread_mutex_lock(&m);  
    done = 1;  
    Pthread_cond_signal(&c);  
    Pthread_mutex_unlock(&m);  
}
```

Parent thread (waiter)

```
void thr_join() {  
    Pthread_mutex_lock(&m);  
    while (done == 0)  
        Pthread_cond_wait(&c, &m);  
    Pthread_mutex_unlock(&m);  
}
```

Any of the following cases can happen while execution.

Case-1: parent creates the child thread but continues running itself-

① parent calls thr_join()

- first locks $m \rightarrow$ checks $done == 0$
- then calls $pthread_cond_wait(&c, &m);$

$\hookrightarrow$ atomically releases m and puts parent to sleep on condition variable c

Now, parent is sleeping (waiting), child can now lock m .

② child runs, print the message "child", and call thr_exit()

- child first locks m
- Holding the lock, it calls $pthread_cond_signal(&c)$

→ it only notifies that one waiting thread can wake up; and does not release the lock.

- child unlocks m

At this point, the signaled parent thread is ready to wake up, but it cannot continue until it successfully successfully relocks m

(3) Parent wakes up.

after being signaled, `pthread_cond_wait()` internally does:

- wake up from the wait queue
- attempt to lock m again
- once the parent successfully acquires m , `pthread_cond_wait()` returns.

Now, parent is awake and holds m again. It rechecks `while(done==0)`
- finds `done==1`, exits the loop.

Then calls `pthread_mutex_unlock(&m)` to release the lock finally, and print the final message "parent".

case-2: the child runs immediately upon creation

- child sets `done=1`, calls `signal` to wake a sleeping thread (but there is none, so it just returns) and is done.

- the parent then runs, calls `thr_join()`, sees that `done==1`, and thus does not wait and returns.

Some observations on the code

① why parent uses a `while` loop instead of just an `if` statement

- due to spurious wakeups - sometimes threads wake up without any signal because of OS scheduler quirks or internal condition variable implementations. So `while` loop ensures safety against false wakeups.
- possibility of multiple waiters and signals - if multiple threads are waiting on the same condition variable, a single signal might only wake one of them - but not necessarily the one for which the condition is actually satisfied. Hence `while` loop ensures correctness.

In short, `while` loop instead of `if` provides a cleaner, safer semantics. It keeps the logic robust even if the code evolves.

(2) why do we need the state variable done? what goes wrong if we do the following instead?

```
1 void thr_exit() {  
2     Pthread_mutex_lock(&m);  
3     Pthread_cond_signal(&c);  
4     Pthread_mutex_unlock(&m);  
5 }  
6  
7 void thr_join() {  
8     Pthread_mutex_lock(&m);  
9     Pthread_cond_wait(&c, &m);  
10    Pthread_mutex_unlock(&m);  
11 }
```

Problem → child runs just after creation, it will signal, but there is no thread asleep on the condition. Then, parent runs, it calls wait and sleeps forever.

Remark: Note that sleeping, waking, etc. are built around the value of the state variable, and hence it's important.

(3) why we need to hold a lock in order to signal and wait? what problems could occur here?

consider the following implementation without lock.

```

1 void thr_exit() {
2     done = 1;
3     Pthread_cond_signal(&c);
4 }
5
6 void thr_join() {
7     if (done == 0)
8         Pthread_cond_wait(&c);
9 }

```

What will happen here?
a subtle race condition.
(think, why?)

Hurt: parent Thread sees
`done == 0`, and before it
actually goes to sleep inside
`Pthread_cond_wait`, the CPU switches
to `thr_exit()` (signal lost?)

Remark: Hold the lock when calling `wait()`, is not just a tip, but
rather mandated by the semantics of `wait()`, because `wait` always

(a) assumes the lock is held when you call it

(b) releases said lock when putting the caller to sleep

(c) re-acquires lock just before returning.